

System Thinking for Forward Deployed Engineers

Finding Extraordinary Problems and Delivering AI Intelligence

Practical Study Guide

IIT Roorkee | Futureense.uni

1. Introduction

After reading this guide, you can apply system thinking to any AI problem, translate an ordinary problem into an extraordinary one using economic levers, and explain the four pillars of intelligence delivery.

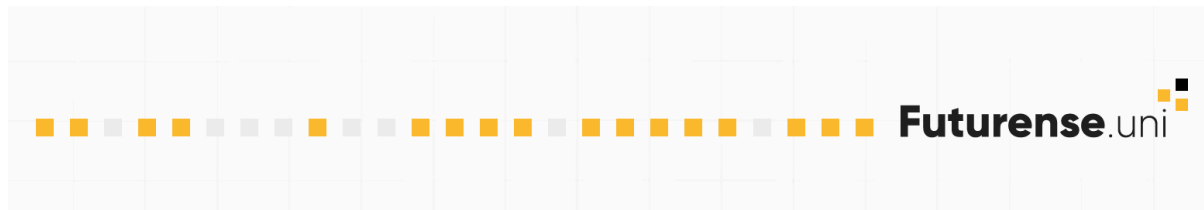
Most engineers learn to solve problems they are given. System thinking is the discipline of questioning whether you are solving the right problem in the first place. In the AI era, this distinction is the difference between building automation (worth cents) and building intelligence (worth billions).

This session is the bridge between the mindset of Session 1 and the technical skills of later sessions. Absorb these frameworks before picking up any tooling.

Topics covered:

- System thinking: definition, purpose, and dual structure
- Four pillars of intelligence delivery: Mental Model, Eval, Generative Force, Sustaining
- The Perplexity case study as a live example of system thinking
- The 6-step framework for translating ordinary problems into extraordinary ones
- Five translation levers: Economic Forces, Data Liquidity, Algorithmic Leverage, Network Effect, First Principles
- JTBD (Job To Be Done) framework and how it applies to AI system design
- Technical debt in AI systems and why it must be designed for, not discovered
- Three mindsets every FDE must carry: Consulting, Research, Economic Value

Key insight: After this guide: You can take any problem statement and, using the translation levers, restate it as an extraordinary problem with economic value, new data, and a unique decision trace.



2. Core Concepts

2.1 What System Thinking Is

System thinking is a mental model you build before you act. It is the discipline of stepping back from a problem to see its full structure: where it came from, what it connects to, what solving it well versus badly looks like, and what creates debt versus value along the way.

System thinking has two jobs. The first is finding the right problem. The second is solving it the right way. Both are required. Solving the wrong problem well is still a failure. Solving the right problem poorly creates technical debt that accumulates over time.

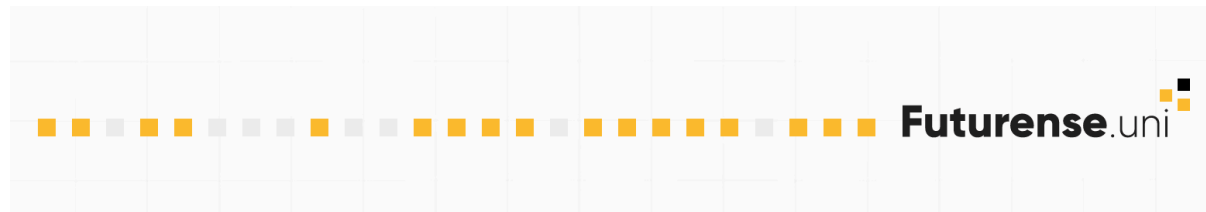
System Thinking = Finding the right problem + Solving it the right way

Key insight: A system thinker does not start with a solution. They start with a first-principle question: what is the customer actually trying to accomplish, and is this problem the real obstacle?

2.2 Four Pillars of Intelligence Delivery

Delivering AI intelligence requires thinking across four layers. Understanding all four is what separates a system thinker from a task executor.

Pillar	What It Is	Why It Matters
Mental Model	A structured way to find the right problem using economic and innovation levers	Without the right problem, all engineering effort goes toward the wrong goal
Eval	Systematic measurement of model capabilities before committing to a technical stack	Choosing the wrong model or stack wastes months and builds technical debt
Generative Force	The four elements that produce intelligence: context, retrieval, memory, feedback	These are the 80% that engineers control and that determine output quality



Sustaining	Guardrails, orchestration, and recovery mechanisms that prevent intelligence from drifting	AI systems degrade over time without explicit maintenance; sustaining is what makes them production-grade
------------	--	---

Master all four pillars before touching any framework or library. These are the design constraints; the code follows.

Watch out: Most engineers jump straight to Generative Force (picking a model, writing prompts) without completing Mental Model or Eval. This is the primary cause of AI projects failing after initial demos succeed.

2.3 Intelligence vs. Tokens

A model produces tokens. That is all a model does. The word after word after word generation that emerges from a language model is just probability distribution sampling.

Intelligence is what those tokens become when they arrive at the customer's reality. Specifically, intelligence is output that is contextualized, conversational, verifiable, and useful, and that enables a decision.

Perplexity does not ship a model. It ships an answer. The answer is verified through citation, structured for decision-making, and relevant to the user's actual question. That is intelligence. The model underneath is interchangeable and gets swapped regularly. The intelligence layer is what users pay for.

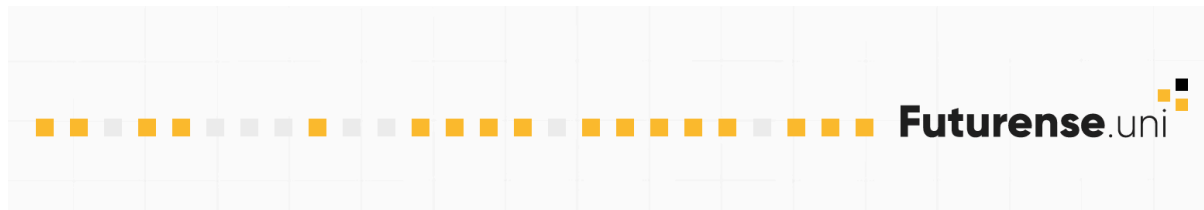
Intelligence = Tokens + Context + Verifiability + Decision-Enabling Usefulness

This distinction matters because it tells you where to focus. Perplexity started 3 years before today's powerful models existed. The engineers who built it did not wait for better models. They built the intelligence layer with whatever models were available and kept the model layer modular. Their USP was never the model. It was the citation-driven, answer-first architecture that no one else had built.

3. Technical Deep Dive

3.1 Technical Debt in AI Systems

Technical debt in traditional software means shortcuts taken today that require cleanup tomorrow: hardcoded values, missing abstractions, skipped tests. In AI systems, technical debt has a second dimension: the probabilistic behavior of the model can corrupt outputs, degrade over time, or produce harm if not explicitly managed.



Three categories of AI technical debt are most common:

- **Model debt:** The model drifts as its training data ages or as new versions replace it. A system that was evaluated on model version A may behave differently on version B without any code change.
- **Context debt:** The data fed to the model at inference time becomes stale, biased, or incomplete. If the documents in a retrieval system are not updated, the model answers from outdated context.
- **Guardrail debt:** Outputs that fall outside acceptable behavior are not caught. This includes hallucinations on edge cases, biased outputs from under-represented training data, and actions taken by agents without appropriate review.

Watch out: Designing only for the happy path is the definition of technical debt in AI engineering. Every system that shipped without explicit failure modes and recovery paths has accumulating debt that will surface in production.

System thinking requires tracking technical debt alongside intelligence. The mental model of your system must include: what can drift, what can fail, what the recovery path is, and how the system degrades gracefully when it fails cleanly.

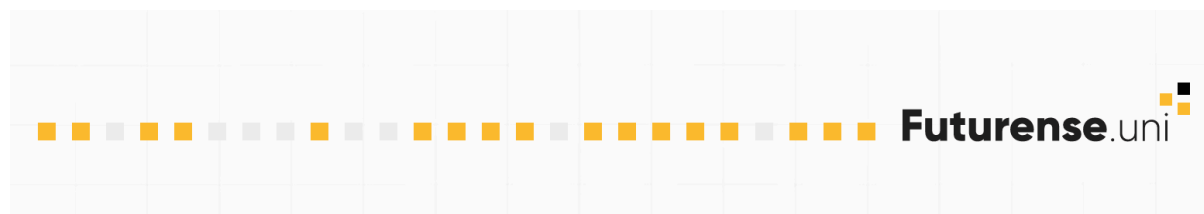
3.2 Transformer Architecture: The Parallelism Insight

The transformer architecture, introduced in the 2017 paper 'Attention Is All You Need' by Ashish Vaswani and colleagues at Google, solved a problem that had blocked NLP progress for years.

Before transformers, language models processed text sequentially using LSTMs (Long Short-Term Memory networks). Each word was processed one at a time. This created two problems. First, training was slow because steps could not be parallelized. Second, context was lost: by the time the model reached word 50, its representation of word 1 had degraded.

Transformers solved both problems with one insight: process all words in parallel and use an attention mechanism to decide which words matter most to each other regardless of their distance in the sequence.

Property	LSTM (Before Transformers)	Transformer
Processing	Sequential: word by word	Parallel: all words at once
Context retention	Degrades over long sequences	Full-sequence attention maintained
Training speed	Slow due to sequential dependency	Fast due to parallelism on GPUs



Context window	Effectively short	Can scale to hundreds of thousands of tokens
Why it matters for FDEs	Limited what problems could be solved	Enables long-document understanding, multi-step reasoning

Transformers enabled large-scale intelligence because parallelism made billion-parameter training economically feasible.

Key insight: A system thinker knows the heart of the technology, not just its definition. The transformer's contribution is parallelism. Everything else (attention, embeddings, scaling laws) built on that single change.

Jensen Huang of NVIDIA saw this shift coming in 2015, a full decade before it became mainstream. He donated GPUs to OpenAI when it was a small research team, understanding that accelerated parallel computation was the prerequisite for everything that followed. That bet turned into NVIDIA's \$4 trillion valuation. That is system thinking applied at an industry level.

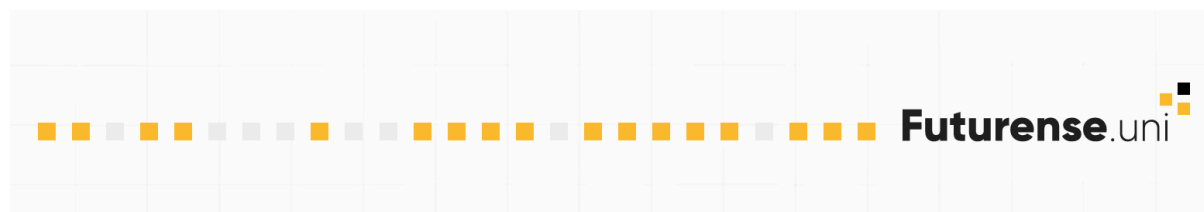
3.3 The Perplexity Architecture: Design by Citation

Perplexity is the best current example of a company that built intelligence, not just an agent. Understanding how their stack works makes the abstract concepts of context, retrieval, and decision enablement concrete.

Perplexity's six-stage pipeline works as follows. First, the user submits a query. Second, the system uses BM25 (text-based search) and dense embedding (semantic search) together to retrieve relevant web content. Third, retrieved content is filtered and re-ranked by source authority and recency. Fourth, a structured prompt is assembled: the user's query plus only the most relevant retrieved content plus citation markers. Fifth, a reasoning model generates the answer using this assembled context. Sixth, the output is formatted with citation links so users can verify every claim.

Their key architectural principle is design by citation. This means no answer is generated without a verifiable source. This is their technical guardrail against hallucination: if no cited source exists for a claim, the answer is withheld.

What Perplexity Does	What Most AI Demos Do
Swaps underlying models regularly	Builds tightly coupled to one model
Builds intelligence in the retrieval and prompt assembly layer	Relies on model quality as the primary differentiator
Verifies every output with citation	Hopes the model is accurate



Solves the 80%: context, retrieval, decision format	Solves the 20%: model selection and API calls
Earns subscriptions for intelligence delivered	Earns nothing because automation does not justify payment

Perplexity's model is interchangeable. Their intelligence layer is not. Build in the right layer.

4. Decision Frameworks

4.1 The 6-Step Framework for Extraordinary Problems

An ordinary problem is what a customer describes when they ask for help. An extraordinary problem is what the customer actually needs, often something they cannot articulate because they are too close to their own situation.

The 6-step framework converts an ordinary problem into an extraordinary one by applying a sequence of translation levers. Each lever forces you to restate the problem from a different angle. By the time you have applied three or more levers, the problem statement has usually changed substantially.

The five primary translation levers are listed below. Apply at least three before declaring a problem extraordinary.

Lever 1: Economic Forces

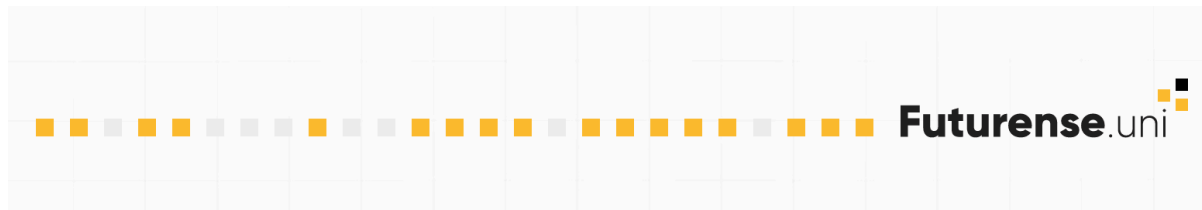
Ask: does this problem have an economic value? Will someone pay for a solution? If solving this problem creates efficiency, productivity gains, new revenue, or cost reduction, it has economic force. If it only creates elegance or completeness, it is a research problem, not a business problem for an FDE.

Test: Can you name a specific customer who would pay for this, and quantify the value to them in dollars or hours?

Lever 2: Data Liquidity

Ask: is there new, dormant, or ignored data that could make this problem solvable in a fundamentally new way? Data that others have overlooked, discarded, or not thought to combine is often the source of extraordinary differentiation.

Examples of data liquidity thinking: Perplexity treated web citations (ignored data) as their primary raw material. Manus treated error logs (discarded data) as their learning signal. A legal AI startup treated case law cross-references (uncombined data) as their competitive moat. A fintech startup combined satellite



land data with government digitization records to create creditworthiness scores for farmers banks had always excluded.

Key insight: Every organization has data they throw away, ignore, or store but never analyze. That data is often worth more than the systems they built to collect it. Ask: what data does the customer produce but not use?

Lever 3: Algorithmic Leverage

Ask: can the decision-making process of this domain be encoded into a unique decision trace that no competitor has? Algorithmic leverage means the way decisions are made, not the code that implements them.

Every organization has an implicit decision logic built from years of experience. Insurance underwriters have a mental model of risk. Lawyers have a mental model of precedent. Farmers have a mental model of soil and yield. This institutional decision knowledge exists in human heads and paper processes. Encoding it as an explicit, AI-assisted decision trace is algorithmic leverage.

Decision and human judgment encoded are the two keywords. Not human-in-the-loop. Encoded: taken out of heads, formalized, and made consistently applicable at scale.

Lever 4: Network Effect

Ask: does solving this problem for one customer make the solution better for the next? Network effects compound value. A system that improves with each additional user, each additional transaction, or each additional data point creates a moat that scales automatically.

For AI systems, network effects often come from feedback loops: each user interaction generates data that improves model performance, which attracts more users. Design this feedback mechanism explicitly.

Lever 5: First Principles

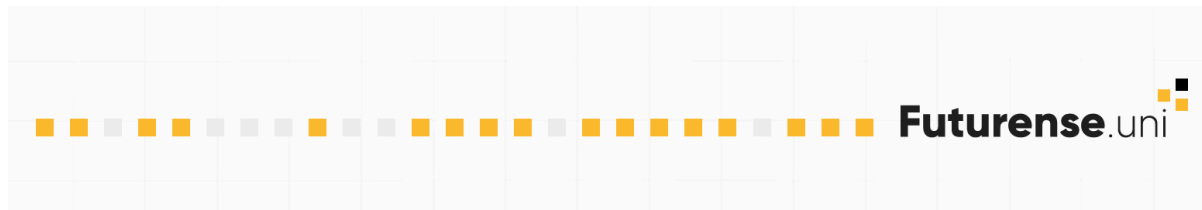
Ask: what is the irreducible truth of what this customer actually wants? Strip away the interface, the history, the habit, and the workaround. Get to the most basic statement of the desired outcome.

Perplexity's first principle: a user who types a question wants an answer, not a list of links to search through. Google habituated users to links for 20 years. Perplexity asked the first-principles question and found an obvious answer that Google had abandoned for commercial reasons. That gap between what exists and what the first principle demands is where extraordinary problems live.

Pair first principles with JTBD (Job To Be Done), described in the next section.

4.2 Job To Be Done (JTBD)

JTBD is a framework invented to understand why customers buy products. The core insight: customers do not buy products for their features. They hire products to do a job that they currently do imperfectly, inconveniently, or expensively.



The milkshake example is the canonical illustration. A fast food chain wanted to sell more milkshakes. Instead of asking customers which flavor they preferred, a researcher asked: when are you buying milkshakes, and what are you trying to accomplish? He discovered that the highest-selling time was morning drive. Customers were not buying milkshakes for taste. They were hiring the milkshake to do three jobs: keep them occupied during a long drive, stay awake, and give them something to sip that would not spill. The competitor in that context was not another milkshake. It was a banana or a granola bar.

The engineer then redesigned the milkshake around those jobs: thicker to sip more slowly, straw size optimized for each sip, cup holder-compatible. Sales increased not because the taste improved but because the product did the job better.

For AI systems: the job is rarely the stated request. An insurance underwriter who asks for faster risk scoring is doing a JTBD of closing more profitable policies with fewer rejections. A lawyer who asks for document summarization is doing a JTBD of spending client time on strategy, not reading. Name the job before writing the prompt.

Key insight: List every job your system does for the user that they would otherwise have to do manually. The longer that list, the more they will pay. JTBD is the checklist of value.

5. Code Walkthroughs

5.1 Encoding a Decision Trace

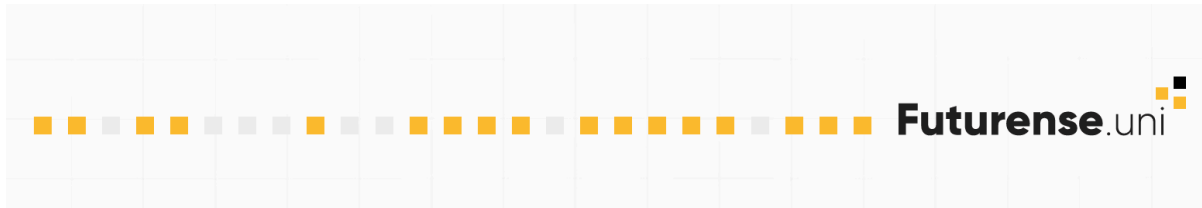
Algorithmic leverage requires making a decision process explicit. This code shows how to represent an underwriting decision trace as structured logic that an LLM can reason within.

Step 1: Define the decision trace as a structured prompt

The decision trace encodes domain knowledge that previously lived in an expert's head. Once explicit, it can be versioned, tested, and improved.

```
import anthropic

DECISION_TRACE = """
Underwriting Decision Framework:
1. Credit risk: Score < 600 = High risk. Score 600-750 = Medium. Score > 750 = Low.
2. Income stability: Employed < 2 years = Flag. Self-employed = Require 3-year revenue history.
3. Debt-to-income: DTI > 0.43 = Reject. DTI 0.35-0.43 = Manual review.
4. Edge case rule: First-time borrowers under 30 with stable employment
```



may qualify despite DTI up to 0.48 with additional guarantor.

"""

```
def underwrite(applicant_data: dict) -> dict:
    client = anthropic.Anthropic()
    prompt = f'{DECISION_TRACE}\nApplicant: {applicant_data}\nDecision:'
    msg = client.messages.create(
        model='claude-haiku-4-5-20251001',
        max_tokens=200,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return {'decision': msg.content[0].text, 'trace': DECISION_TRACE}
```

Step 2: Test the decision trace against a sample applicant

```
applicant = {
    'credit_score': 720,
    'employment_years': 3.5,
    'dti': 0.40,
    'age': 27,
    'employment_type': 'employed'
}
```

```
result = underwrite(applicant)
print(result['decision'])
```

Output:

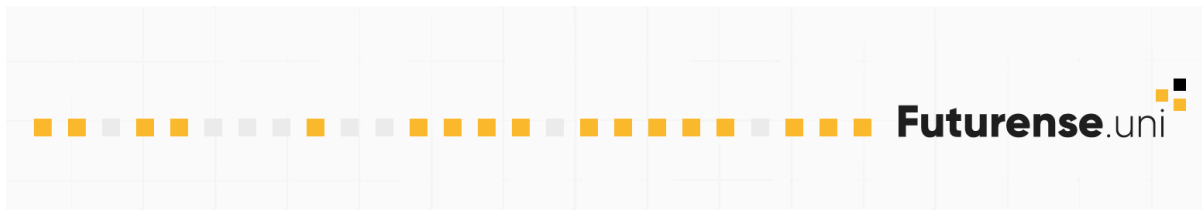
Manual review required. Credit score is acceptable (720, Medium risk). Employment stable at 3.5 years. DTI of 0.40 falls in the manual review band (0.35-0.43). Recommend reviewing with guarantor option given applicant age.

This output is verifiable, traceable to specific rules, and provides a clear recovery path (guarantor option). It is intelligence, not just a response. The decision trace is the algorithmic leverage: it encodes rules that were previously implicit in an underwriter's judgment.

5.2 Data Liquidity: Finding Gold in Discarded Data

Data liquidity means using data that was previously ignored, discarded, or not thought to combine. This code demonstrates a simple but powerful pattern: combining two data sources that no one previously linked.

Step 3: Build a context from two separate data sources



This simulates the Harvey legal AI approach: linking case laws that were never connected, then reasoning across the combined context.

```
import anthropic
import json

# Source 1: Supreme Court ruling (usually stored, never linked)
supreme_court_ruling = {
    'case': 'Sharma v State 2019',
    'principle': 'Digital records admissible with chain of custody documentation'
}

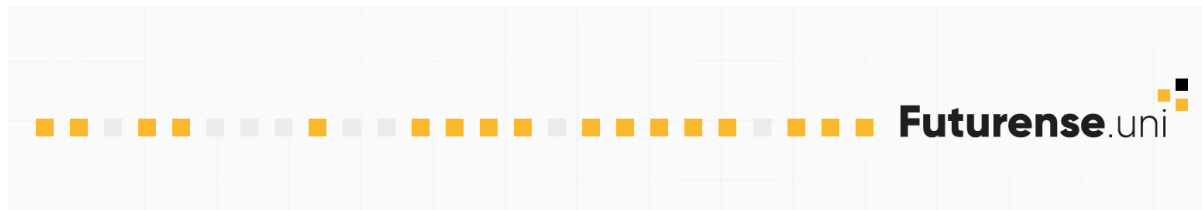
# Source 2: High Court precedent (different system, never compared)
high_court_precedent = {
    'case': 'Rajan v Municipality 2021',
    'principle': 'Electronic timestamps sufficient for chain of custody'
}

def generate_legal_strategy(new_case: str) -> str:
    client = anthropic.Anthropic()
    context = json.dumps([supreme_court_ruling, high_court_precedent])
    prompt = f'Relevant precedents: {context}\nNew case: {new_case}\nStrategy:'
    msg = client.messages.create(
        model='claude-haiku-4-5-20251001',
        max_tokens=300,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return msg.content[0].text
```

Output:

Based on linked precedents: Digital evidence admissible. Use Sharma v State 2019 to establish admissibility, then apply Rajan v Municipality 2021 to argue that electronic timestamps satisfy chain of custody. Combined, these precedents reduce documentation burden by ~40 hours of manual research.

Key insight: The value is not in either ruling individually. It is in the connection between them that no lawyer had time to build manually. That connection is data liquidity in action.



6. Case Studies

Case Study 1: Harvey AI (Legal Intelligence)

Problem: Legal research for Supreme Court cases requires a lawyer to manually trace precedents across hundreds of cases across different courts. This takes 6 to 8 weeks of associate time per case.

Why this needs new data: Case laws exist in separate databases (Supreme Court, High Courts, Tribunals) that were never cross-linked. The data existed. The connections did not.

Right approach: Harvey built a system that indexes all case law across courts, identifies citations between them, and constructs a linked graph of precedents. When a new case is filed, the system traces the relevant graph automatically and produces a strategy document with linked citations. What took 6 weeks takes 2 days.

Wrong approach: Build a chatbot that lets lawyers ask questions about case law using a single court's database. The chatbot answers individual questions but misses the cross-court connections that form the strongest arguments.

Result: 40x reduction in research time. The differentiation is not the language model. It is the cross-court citation graph that no competitor had built.

Case Study 2: Agri-Fintech Startup (Financial Inclusion)

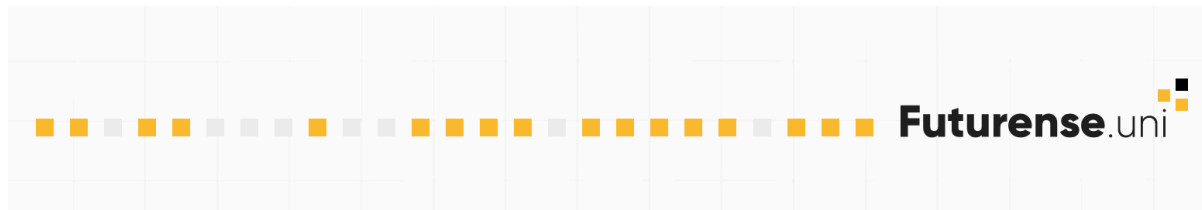
Problem: Banks exclude rural farmers from loans because traditional creditworthiness metrics (credit score, salary slips, employment history) do not apply to agricultural borrowers.

Why this is extraordinary: 60% of India's population is engaged in agriculture. Banks had a rule-based exclusion that was economically irrational: the risk was assessable, but no one had built the assessment tool.

Right approach: The startup combined three previously separate data sources: government land ownership records (newly digitized), satellite geospatial data on land productivity, and historical weather patterns for each region. These three sources together could predict farm productivity and therefore loan repayment probability. The startup built a creditworthiness model from this combined data and licensed it to banks as a lending-decision tool.

Wrong approach: Ask banks to provide their existing customer data and build a model from that. This excludes the same customers the banks already exclude and creates no new market.

Result: The startup enabled lending to a population banks could not previously assess. They grew to unicorn valuation not by improving existing banking but by creating a new lending market from dormant government data.



Key insight: The data existed in government records for years. No one combined it with satellite imagery and weather history. The value was in the combination, not in any single source.

7. Common Mistakes

Mistake 1: Stating the problem before applying levers

The first statement of a problem is almost always the ordinary framing. It describes a symptom, not a root cause. It mirrors what the customer said, not what they need. Writing the problem statement and submitting it without running it through at least three translation levers means you are building automation, not intelligence.

Mistake 2: Confusing algorithmic leverage with code

Algorithmic leverage is not about writing a more sophisticated algorithm in code. It is about encoding a decision trace that reflects domain expertise. The question is: what decisions does this domain require, and how are those decisions currently being made by humans? Encode that logic as a structured prompt or constraint set. The code that implements it is a detail.

Mistake 3: Ignoring network effects

The most defensible AI systems improve with use. Each user interaction, each correction, each new data point strengthens the system. If you design a system that produces the same quality output on day 1 and day 365, you have built a product, not a compounding advantage. Always ask: where does the feedback loop live, and how does it make the system stronger over time?

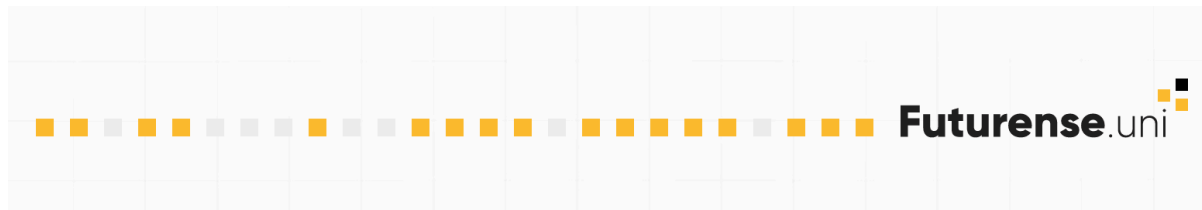
Mistake 4: Solving for the feature, not the job

The customer asks for faster ticket resolution. The JTBD is never having to file a ticket. The customer asks for better search results. The JTBD is arriving at a decision in seconds, not minutes. The stated request and the actual job are almost always different. Building what was asked for without doing JTBD analysis means building a half-solution that will be replaced when someone builds the full one.

Mistake 5: Treating technical debt as optional

AI systems that fail in production do not usually fail because the model was wrong. They fail because data drifted, context became stale, or a guardrail was missing. These failures were predictable at design time. System thinking requires building technical debt inventory alongside the feature spec. Every design decision that creates future fragility is technical debt. Name it, document it, and create a maintenance schedule for it at design time.

Mistake 6: Anchoring to the current model



Perplexity swaps its underlying model regularly. The reason is architectural: they built their intelligence in the retrieval and context layer, not the model layer. If you build tightly coupled to a specific model, every model upgrade becomes a migration project. The 80% that you control (data, context, evals, failure modes) should be model-agnostic. The model is a plug-in, not a foundation.

8. Interview Preparation

These questions appear in FDE and senior AI engineering interviews. Each tests system thinking, not just technical knowledge.

Q1: A client asks you to build an AI that reduces support ticket resolution time by 50%. How do you approach this?

Answer: First, I would apply JTBD. The client's stated job is faster ticket resolution. The actual job is fewer tickets. I would ask: what is driving ticket volume? If 60% of tickets fall into 8 categories, the intelligent solution is eliminating the root cause of those 8 categories, not processing them faster. I would then apply data liquidity: what data does the client generate that, if analyzed, would predict which users are about to file a ticket? With that data, I can build a proactive help system that intercepts the problem before the ticket is filed. The extraordinary framing shifts the metric from resolution time to ticket volume reduction.

What they test: Whether you challenge the problem statement or accept it, and whether you think upstream or just downstream.

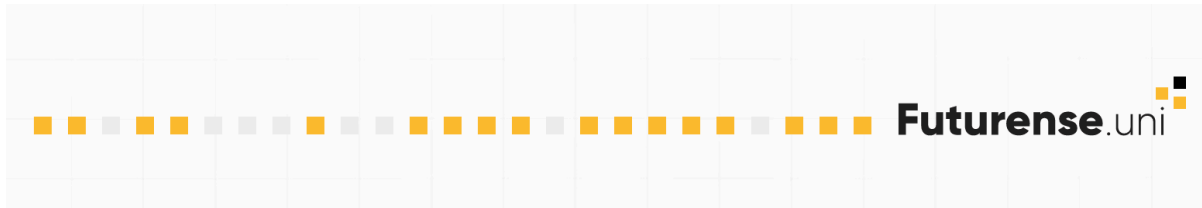
Q2: What is the difference between a decision trace and an if-then-else block?

Answer: An if-then-else block encodes a known case explicitly. It handles what you already know. A decision trace encodes the reasoning process of a domain expert, including how they handle ambiguity, weight competing factors, and reach conclusions under uncertainty. An LLM can reason within a decision trace to handle cases that were never explicitly programmed. The insurance underwriter's mental model of a borderline case, the lawyer's judgment on a novel precedent, the analyst's intuition on an unusual data pattern, these can be expressed as decision traces and made consistently applicable. The if-then-else block cannot do this.

What they test: Whether you understand what AI uniquely enables versus what traditional code already does.

Q3: How do you prevent technical debt accumulation in an AI system?

Answer: At design time, I build a debt inventory alongside the feature spec. For each design decision, I ask: what fails if this assumption changes? Model drift, context staleness, guardrail gaps, and feedback loop failures are all predictable. I document each one with a monitoring mechanism and a maintenance



schedule. For context, I set a data freshness TTL and automate re-ingestion. For models, I run the eval harness against new model versions before any upgrade. For guardrails, I maintain a failure mode log and review it monthly. Technical debt in AI is not discovered in production; it is designed at specification time and either managed or ignored.

What they test: Whether you think about production before you have written a line of code.

Q4 (Code challenge): Write a function that runs a simple JTBD analysis using an LLM.

```
import anthropic

def jtbd_analysis(problem_statement: str) -> dict:
    client = anthropic.Anthropic()
    prompt = f"""
    Apply Job-To-Be-Done analysis to this problem statement.
    Problem: {problem_statement}

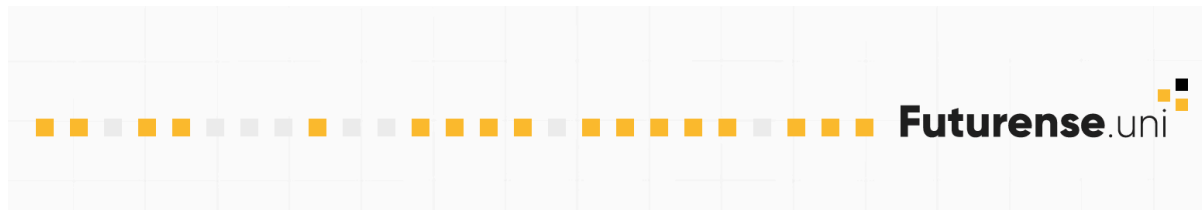
    Answer these three questions:
    1. What is the customer's stated job (what they asked for)?
    2. What is the actual underlying job (what they need)?
    3. What does the extraordinary solution look like?

    Be specific and concrete. One sentence per answer.
    """
    msg = client.messages.create(
        model='claude-haiku-4-5-20251001',
        max_tokens=300,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return {'analysis': msg.content[0].text, 'input': problem_statement}

result = jtbd_analysis(
    'Build AI to close support tickets faster'
)
print(result['analysis'])
```

Output:

1. Stated job: Process support tickets in less time. 2. Actual job: Customers want their problems solved without friction, not faster ticket handling. 3. Extraordinary solution: Predict which users will file tickets based on product usage patterns and deliver contextual help before they reach the filing step, eliminating 60% of tickets at source.



9. Practice Problems

9.1 MCQs

Q1. An engineer receives this problem statement: 'Build AI to grade essays faster.' Applying system thinking, what is the most likely extraordinary reframing?

- A) Build AI that grades essays in 5 seconds instead of 5 minutes.
- B) Build AI that provides real-time writing feedback while students draft, eliminating grading as a bottleneck by improving first drafts.
- C) Build AI that assigns numerical scores automatically using rubric matching.
- D) Build AI that summarizes teacher comments into one-sentence feedback.

Answer: B. Option B reframes the problem from a downstream activity (grading) to an upstream intervention (improving drafts). This is the JTBD insight: students do not want faster grades, they want better writing. Options A and C solve the stated problem. Option D reduces effort without changing the fundamental flow.

Q2. Which of the following is the best example of data liquidity?

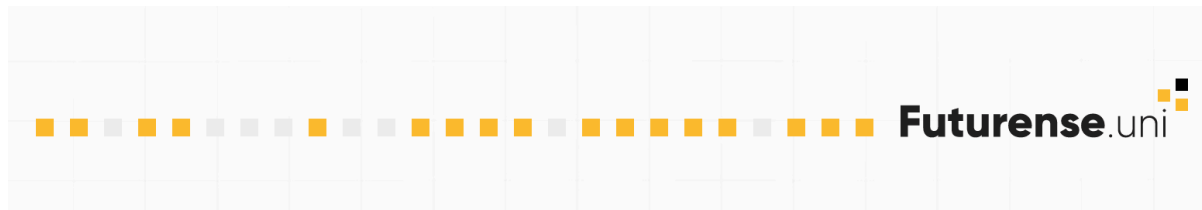
- A) Using a larger training dataset to improve model accuracy.
- B) A legal AI startup linking Supreme Court and High Court rulings that were never cross-referenced, creating a new precedent graph.
- C) Adding more user feedback to a fine-tuning dataset.
- D) Increasing the context window to include more documents in a RAG pipeline.

Answer: B. Option B is correct. Data liquidity means using data that was previously ignored, discarded, or not thought to combine. The cross-referencing of court rulings is data that existed but was never linked. Options A, C, and D improve existing pipelines but do not identify new or dormant data.

Q3. What is algorithmic leverage, according to the session?

- A) Using a faster sorting algorithm to improve query performance.
- B) Encoding domain expertise and decision-making logic into an AI system as a unique decision trace.
- C) Using more parameters in the model to improve reasoning capability.
- D) Switching from rule-based systems to machine learning.

Answer: B. Option B is correct. Algorithmic leverage is about encoding the decision trace of domain experts, not about code optimization or model size. The HDFC underwriting algorithm is different



from Bajaj Finance's because their accumulated human judgment differs. Encoding that uniquely is the leverage.

Q4. System thinking has two primary purposes. Which pair is correct?

- A) Writing better prompts and selecting the right model.
- B) Finding the right problem and solving it the right way.
- C) Reducing hallucination and increasing context window efficiency.
- D) Building agents and maintaining their performance over time.

Answer: B. Option B is correct. System thinking as defined in this session has exactly two jobs: identify the extraordinary problem, and design the solution without accumulating technical debt. Options A, C, and D are implementation concerns, not system thinking definitions.

Q5. Perplexity regularly swaps its underlying model without disrupting its core product. Why is this possible?

- A) They have a proprietary model that is version-stable.
- B) Their intelligence layer is built in the retrieval, prompt assembly, and citation layer, not in the model layer.
- C) They use model-agnostic APIs that abstract all differences.
- D) They only use open-source models that are freely interchangeable.

Answer: B. Option B is correct. Perplexity's USP is the citation-driven answer architecture. The model is a commodity component. When they swap models, the intelligence layer (retrieval, re-ranking, structured prompt assembly, citation enforcement) stays constant. This is the correct architectural lesson.

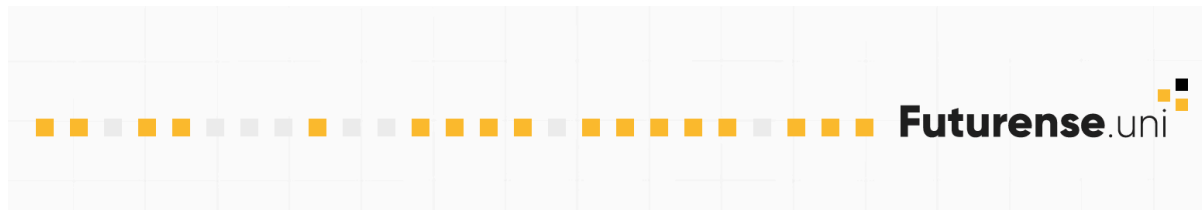
Q6. What was the key innovation in transformer architecture that enabled large language models?

- A) Using more layers in the neural network.
- B) Replacing gradient descent with a faster optimization algorithm.
- C) Processing all words in parallel using attention, enabling efficient training and full-sequence context.
- D) Adding memory modules that store previous conversations explicitly.

Answer: C. Option C is correct. The transformer's core contribution is parallel processing enabled by the attention mechanism. This replaced sequential LSTM processing, made billion-parameter training feasible on GPUs, and maintained full-sequence context without degradation over long documents.

Q7. A startup builds an AI system that gets better at predicting loan defaults as more loans are processed. What economic lever is this?

- A) Data Liquidity
- B) Algorithmic Leverage



- C) First Principles
- D) Network Effect

Answer: D. Option D is correct. Network effects occur when each additional user or transaction makes the system better for all future users. A loan prediction system that improves with each processed loan has a self-reinforcing feedback loop. This is a network effect. Data liquidity (A) refers to new or dormant data sources. Algorithmic leverage (B) refers to decision trace encoding.

Q8. An FDE is presenting to a bank customer about AI underwriting. Which of the three FDE mindsets should dominate this interaction?

- A) Research mindset: explaining the transformer architecture and model selection process.
- B) Consulting mindset: explaining the agent architecture and tool-calling framework.
- C) Economic value mindset: showing how a new decision trace enables the bank to approve 25% more loans profitably.
- D) Technical mindset: demonstrating the accuracy metrics on the internal eval dataset.

Answer: C. Option C is correct. When facing a customer, the economic value mindset dominates. The customer is not paying for technology; they are paying for business outcomes. Showing 25% more profitable loan approvals is the economic value argument. Options A and D are internal technical discussions. Option B describes a feature without connecting it to business impact.

Q9. Read this prompt structure and identify which translation lever it applies: 'What data does your organization generate today but never analyzes? What would be possible if that data were connected to your current decision systems?'

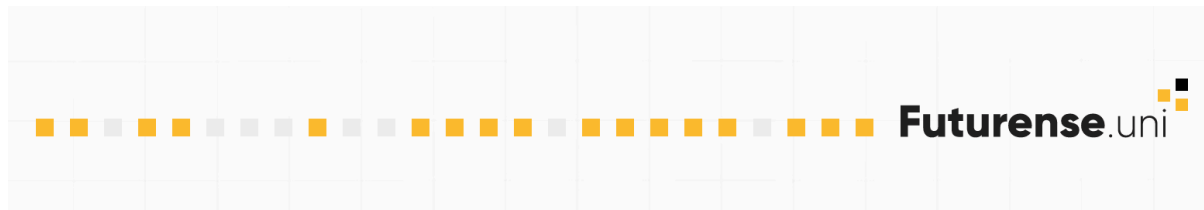
- A) Algorithmic Leverage
- B) Network Effect
- C) First Principles
- D) Data Liquidity

Answer: D. Option D is correct. This prompt asks specifically about existing but unused data, which is the definition of data liquidity. It seeks to identify dormant or discarded data that could change the economics of a problem.

Q10. Which of the following best describes a technical debt situation specific to AI systems?

- A) A developer commits code without writing unit tests.
- B) A retrieval system's document corpus is not updated after the product launches, so the model answers from stale context six months later.
- C) An API endpoint does not have proper error handling.
- D) A database schema was not normalized before data was inserted.

Answer: B. Option B is correct. Context staleness is a form of AI-specific technical debt. The system was designed without a data freshness mechanism. Six months after launch, the model answers from



outdated context, producing wrong outputs. This failure was predictable at design time and is a system thinking failure.

9.2 Assignments

Assignment 1: Apply Three Translation Levers

Goal: Practice converting an ordinary problem into an extraordinary one.

Scenario: A hospital chain asks you to build AI that reduces patient wait times in the emergency department. The average patient waits 90 minutes before seeing a doctor.

Requirements:

- Write the ordinary framing: what the hospital asked for, taken at face value.
- Apply Lever 1 (Economic Forces): what is the economic value of reducing wait times? Who pays and what do they pay?
- Apply Lever 2 (Data Liquidity): what data does the hospital generate that could predict patient flow, triage priority, or staff bottlenecks?
- Apply Lever 3 (JTBD): what is the actual job a patient in the ED is trying to hire the hospital to do?
- Write the extraordinary reframing: a problem statement that incorporates your findings from all three levers.

Submit: Five paragraphs, one per requirement. The extraordinary reframing should be 3-5 sentences and measurably different from the ordinary framing.

Strong answer: The extraordinary reframing shifts from 'reduce wait time' to 'predict patient deterioration before it escalates, enabling triage before the patient self-presents.' It identifies historical triage data and wearable-device vital readings as data liquidity. It quantifies economic value as reduced readmission costs. Weak answer: Reframes as 'reduce wait time from 90 to 60 minutes using AI scheduling' without changing the fundamental problem.

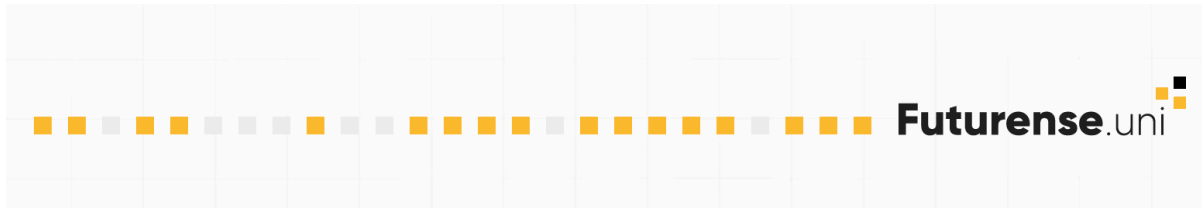
Assignment 2: Build and Test a Decision Trace

Goal: Encode a domain decision as a structured prompt and verify it produces correct outputs on test cases.

Scenario: A customer support team escalates tickets to senior agents when certain conditions are met. The current rules are: ticket age > 48 hours, customer tier = premium, or keyword 'legal' or 'refund > 5000' appears in the ticket.

Requirements:

- Encode the escalation decision trace as a structured system prompt.



- Write a Python function that takes a ticket description and returns a structured decision with reasoning.
- Test with at least 4 tickets: one clear escalation, one clear non-escalation, one edge case that should escalate, one that should not.
- For the edge case, show how the decision trace handles ambiguity (does it escalate, and why?).

Submit: Python code with 4 test case outputs. Each output must include the decision and a one-sentence reason.

Grading: Strong submission shows reasoning traces in the output, handles the edge case explicitly, and uses structured output format. Weak submission only returns yes/no without explanation.

Assignment 3: JTBD Analysis on a Real Problem

Goal: Apply JTBD to a real problem from your current work or a domain you know.

Choose a problem you or your team is currently solving or considering. It can be in any domain.

Requirements:

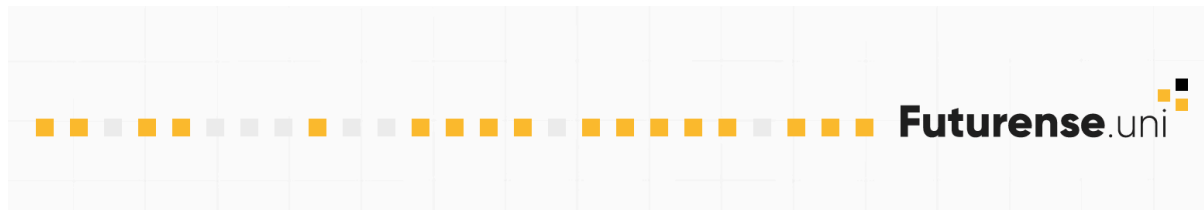
- State the problem as it was originally presented (the ordinary framing).
- Apply JTBD: what is the stated job, and what is the actual underlying job the customer is trying to accomplish?
- Identify one piece of dormant or ignored data in this domain that, if used, would change the solution.
- Write the extraordinary reframing in 3-5 sentences.

Submit: 400-600 words. Use your actual work context. Generic or hypothetical answers score lower than specific, grounded ones.

10. Further Learning

Work through these in the order listed. They build on each other.

- JTBD original framework: Clayton Christensen's 'Competing Against Luck'. The milkshake story is in Chapter 2. Read the first 3 chapters for the complete framework.
- Perplexity architecture deep dive: search for Perplexity's technical blog posts on their retrieval pipeline. Specifically look for posts on their BM25 plus dense embedding hybrid approach.



- Transformer attention mechanism: The original 'Attention Is All You Need' paper (Vaswani et al., 2017). Read the abstract and Section 3. The KV cache optimization is a follow-on concept worth exploring.
- Satya Nadella at World Economic Forum 2024: search for the full interview. His statement on decision traces and algorithmic leverage is the source material for Lever 3.
- Harvey AI case study: search for Harvey AI legal research and read their founding story and early investor materials. This is the data liquidity case study in full detail.
- First Principles thinking: Elon Musk's battery cost example is the clearest illustration. In his original explanation, he asks why batteries are assumed to cost \$600/kWh and then breaks the cost down to raw materials, finding that raw material cost is \$80/kWh. The gap is the opportunity.
- Eval design: The RAGAS framework (Retrieval Augmented Generation Assessment) provides a structured approach to evaluating RAG-based AI systems on faithfulness, answer relevance, and context recall.

11. Cheatsheet

Quick reference for design reviews, problem framing sessions, and interview preparation.

Key Definitions

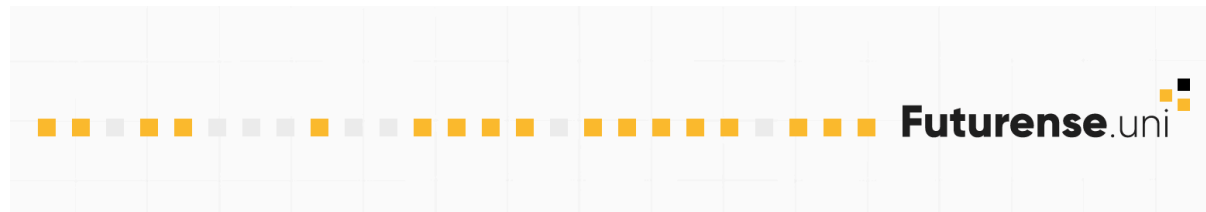
Term	Definition
System thinking	Building a mental model before acting: find the right problem, then solve it the right way
Translation lever	A lens applied to an ordinary problem to reveal its extraordinary version
Data liquidity	Using data that is new, dormant, ignored, or previously uncombined
Algorithmic leverage	Encoding domain decision logic as a unique decision trace no competitor has
Network effect	A system that improves with each additional user or transaction

First principles	The irreducible truth of what the customer actually wants, stripped of habit and workaround
JTBD	Job To Be Done: the underlying job a customer hires a product to accomplish
Decision trace	The explicit encoding of how a domain expert reaches decisions, including ambiguity handling
Technical debt (AI)	Design decisions that create future fragility: model drift, context staleness, missing guardrails
Intelligence	Tokens that arrive at the customer's reality as contextualized, verifiable, decision-enabling output
Four pillars	Mental Model, Eval, Generative Force, Sustaining: the complete intelligence delivery architecture

These definitions are the vocabulary of system thinking. Use them precisely in customer conversations.

Translation Lever Application Order

Step	Lever	Question to Ask
1	Economic Forces	Will someone pay for this? Can I quantify the value in dollars or hours saved?
2	Data Liquidity	What data exists but is ignored? What can I combine that no one has combined?
3	Algorithmic Leverage	What decision process can I encode? What domain expertise can I make explicit?
4	Network Effect	Does this improve with use? Where is the feedback loop?



5	First Principles + JTBD	What is the irreducible truth? What job is the customer actually hiring this to do?
---	-------------------------	--

Apply at least 3 levers before calling a problem extraordinary. The more levers that apply, the harder the solution is to copy.

Formulas

System Thinking = Finding the right problem + Solving it the right way

Intelligence = Tokens + Context + Verifiability + Decision-Enabling Usefulness

Technical Debt = Sum of design decisions made without failure modes and recovery paths

Algorithmic Leverage = Encoded domain decision trace that no competitor possesses

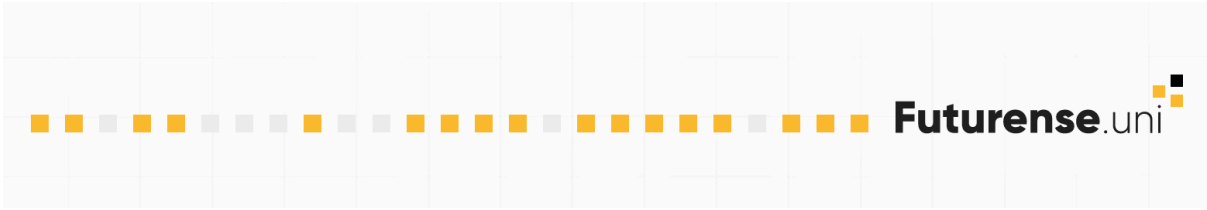
Extraordinary Problem = Ordinary problem + 3+ translation levers applied

Golden Rules

- Never submit the first statement of a problem as the final one. Apply levers first.
- The model is interchangeable. The intelligence layer is the product.
- Data others throw away is often worth more than data everyone collects.
- Decision traces encode expertise. Code implements decision traces. The order matters.
- Every AI design decision that creates future fragility is technical debt. Name it at design time.
- JTBD before writing a prompt. Know the actual job before designing the solution.
- Economic value is the validator of extraordinary problems. Research has no place in an FDE's delivery.
- System thinking cannot be learned from syntax. It must be practiced on real problems, weekly.

Three FDE Mindsets

Mindset	When to Apply	What It Produces
Research	Internally, when understanding the technology and its limits	Deep capability knowledge, architectural judgment
Consulting	When communicating with customers and stakeholders	Clear problem framing, executive presence, trust



Economic Value	When presenting solutions and closing proposals	Business outcomes, ROI arguments, customer payment
----------------	---	--

All three mindsets are needed. The error is applying the wrong one at the wrong time: using research language with a CFO, or consulting language when the tec